

Introduction to React

Lesson 1: What is React and Why Use It?

Learning Objectives

- Understand what React is and what problem it solves.
- Understand why React is called a library and how it differs from a framework.
- Learn about Meta and the React open-source community.
- Identify important React features and real-world use cases.
- Understand why React is popular for modern user interfaces.

What is React?

React is a JavaScript library for building user interfaces. It helps developers create interactive screens from reusable pieces called components. A component can be as small as a button or as large as a complete page.

React is maintained by Meta and an open-source community. Its component-based approach makes it easier to organize, reuse, and maintain UI code as an application grows.

React Library vs Framework

React mainly focuses on the user interface. A library usually solves a focused problem, while a framework generally provides a broader application structure and additional conventions. React can be combined with other tools for routing, API communication, state management, forms, and styling.

Important React Features

- **Component-based architecture:** Build the UI from independent, reusable pieces.
- **Declarative UI:** Describe what the UI should look like for the current data or state.
- **Reusable components:** Create a component once and use it with different data.
- **State-driven updates:** When relevant state changes, React updates the UI.
- **JSX:** A syntax commonly used in React projects to write markup close to component logic.
- **One-way data flow:** Data can be passed from parent components to child components using props.
- **Large ecosystem:** React can be combined with libraries and frameworks for larger applications.

How React Uses Components

Consider an LMS application. Instead of putting every part of the interface in one large file, the application can be divided into components such as Navbar, Sidebar, CourseCard, LessonList, Quiz, ProgressBar, and Profile. Each component can focus on one part of the UI.

Example structure:

App ■■■■ Navbar ■■■■ Sidebar ■■■■ CourseList ■ ■■■■ CourseCard ■ ■■■■ CourseCard ■■■■
LessonList ■ ■■■■ LessonItem ■■■■ Footer

Why React is Popular

- Reusable components reduce repeated UI code.
- Its declarative model makes dynamic interfaces easier to reason about.
- It can scale from small interfaces to large applications.
- It has extensive documentation and a large developer ecosystem.
- React skills can also be applied to native application development through related tools such as React Native.

Real-World Use Cases

- Learning Management Systems: courses, lessons, quizzes, progress tracking, and dashboards.
- E-commerce applications: product listings, carts, filters, and checkout flows.
- Admin dashboards: tables, charts, reports, and forms.
- Social and interactive applications: feeds, comments, notifications, and profiles.
- SaaS applications: account areas, settings, analytics, and data-driven interfaces.

Example: A Reusable React Component

The following component receives a name through props and displays a welcome message.

```
function Welcome({ name }) { return Welcome, {name}!; } export default Welcome;
```

The same component can be reused with different values:

```
function App() { return ( ); } export default App;
```

Code Example: Interactive Counter

```
import { useState } from "react"; function Counter() { const [count, setCount] =  
useState(0); return ( React Counter You clicked {count} times setCount(count + 1)>> Click  
me ); } export default Counter;
```

Code Explanation

The `useState` hook creates a state value named `count` with an initial value of 0. When the button is clicked, `setCount` updates the value. React then renders the component again so the screen reflects the latest count.

Key Takeaways

- React is a JavaScript library focused on building user interfaces.
- React applications are built from reusable components.
- Components are JavaScript functions that return UI markup.

- React commonly uses JSX to describe UI.
- State changes can cause React to update the displayed UI.
- React is flexible and can be combined with other tools as an application grows.

Further Learning

Official React Quick Start: react.dev/learn

Official React Components Guide: react.dev/learn/your-first-component

Practice Questions

- 1 What is React primarily used for?
- 2 Why is React described as a library?
- 3 What is a component?
- 4 How does component reusability help a large application?
- 5 What happens to the displayed UI when relevant React state changes?
- 6 Name three real-world applications that can use React.